RICHFIELD GRADUATE INSTITUTE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY

MOBILE APP DEVELOPMENT 700

SMART PANTRY MANAGER

A Java Android Application for Reducing Food Waste Through Strict Ingredient Matching

Student name:Jacueline Mama Motapanyane

Student ITS number: 402307174

Qualification / year / semester: BSc In IT, 2026,2

Submission date: 23 October 2026

Table of Contents

- 1. Introduction
- 2. System Design
- 3. Database Design
- 4. Implementation Overview
- 5. Testing and Evidence
- 6. Challenges and Solutions
- 7. Conclusion and Reflection
- 8. References

1. Introduction

Food waste is a practical household problem: ingredients may remain unused until they expire, while people may assume that cooking a meal requires buying additional groceries. Smart Pantry Manager

is designed to help users make better use of ingredients they already have by maintaining a digital pantry and identifying recipes that can be prepared from the available stock.

The application is intended for people who want a simple way to record ingredients, manage quantities and discover suitable meals without relying on a shopping trip. It is designed as a native Android application written in Java, using Android Studio concepts such as Activities, layouts, Intents, adapters and persistent local storage.

The central business rule is strict recipe matching. A recipe is suggested only when every required ingredient is present in the pantry in at least the required quantity. A recipe that is missing even one required ingredient must not appear in the main suggested-recipes list.

1.1 Aim and objectives

- Provide create, read, update and delete (CRUD) operations for pantry ingredients.
- Display pantry records in a dynamic list.
- Store a collection of at least 15 recipes with required ingredients and preparation steps.
- Show only recipes that satisfy the strict ingredient and quantity rule.
- Provide a recipe detail view and a settings/profile screen.
- Retain pantry data when the application is closed and reopened.

1.2 Scope

The application focuses on the user's own pantry and recipe matching. Location services, Google Maps, payment processing and Play Store publication are outside the assignment scope.

2. System Design

The planned user journey begins at the pantry list. The user can add or edit an ingredient, open suggested recipes, select a matching recipe to see its details, or open settings. Navigation between screens should use Android Intents (or a consistent single-Activity navigation design).

2.1 Screen flow

Pantry List → Add/Edit Ingredient → Save → Pantry List → Suggested Recipes → Recipe Detail. Pantry List → Settings. The suggested list is recalculated from the current pantry data whenever it is opened or refreshed.

2.2 Screen responsibilities

- Pantry List: display ingredient name, quantity, unit and optional expiry date; offer add, edit and delete actions.
- Add/Edit Ingredient: validate the ingredient name and quantity, then save the record.
- Suggested Recipes: show only recipes whose complete ingredient requirements are satisfied; display a clear empty-state message if none qualify.
- Recipe Detail: display the recipe name, required ingredients and preparation method.
- Settings: provide a small preference such as expiry-alert preference or unit display preference.

3. Database Design

For a straightforward local Android implementation, SQLite is a suitable choice because it stores data on the device and is accessible through Android's SQLiteOpenHelper API. The final report must describe the database actually used by the submitted application; if Firebase or PostgreSQL was implemented instead, revise this section accordingly.

3.1 Proposed data model

PantryItem: id (primary key), name, quantity, unit, expiryDate (nullable). Recipe: id (primary key), name, method. RecipeIngredient: id (primary key), recipeId (foreign key), ingredientName, requiredQuantity, unit. AppSetting: settingKey (primary key), settingValue.

The RecipeIngredient table separates recipes from their ingredients, allowing each recipe to have multiple ingredient rows. RecipeIngredient.recipeId references Recipe.id. PantryItem records represent the current stock available to the user.

3.2 Data integrity and persistence

Input validation should reject a blank ingredient name and a quantity that is not a valid positive number. Database operations should use parameterised values rather than concatenating user input into SQL. Pantry data must be read from persistent storage when the list screen opens so that records remain available after the app process is closed.

4. Implementation Overview

4.1 Java and Android components

Activities represent the main screens, XML layouts define the visual interface, and Intents pass the selected record identifier between screens. A RecyclerView with a custom Adapter is appropriate for presenting pantry ingredients or recipe results. The adapter binds each database record to a row and exposes actions for editing or deleting records.

4.2 Strict-matching algorithm

For every recipe, the application checks each required ingredient against the current pantry. Ingredient names should be normalised before comparison by trimming whitespace and using a consistent case. Simple singular/plural handling can be added to reduce avoidable mismatches. A recipe qualifies only if every required ingredient is found and the available quantity is at least the required quantity. A partial match is rejected.

Pseudocode: for each recipe, set qualifies = true; for each required ingredient, find a matching pantry item; if no match exists or available quantity is too small, set qualifies = false and stop checking that recipe; add the recipe to suggestions only if qualifies remains true.

4.3 CRUD behaviour

- Create: validate the form and insert the new ingredient into the database.
- Read: query all pantry ingredients and display them in the list adapter.
- Update: load the selected ingredient by ID, allow changes, and update the stored row.
- Delete: remove the selected record and refresh the list.

5. Testing and Evidence

Test ID	Test scenario	Expected result	Actual result / evidence
T1	Add a valid pantry ingredient	Ingredient is saved and shown in the pantry	[Run test; record result]
T2	Submit a blank ingredient name	Validation message appears; no blank	[Run test; record result]

		record saved	
T3	Edit an existing ingredient	Updated value is displayed and persisted	[Run test; record result]
T4	Delete an ingredient	Deleted record no longer appears	[Run test; record result]
T5	Recipe has every ingredient in sufficient quantity	Recipe appears in suggestions	[Run test; record result]
T6	Recipe is missing one required ingredient	Recipe is excluded from suggestions	[Run test; record result]
T7	Close and reopen the app	Previously saved pantry data remains	[Run test; record result]
T8	No recipes qualify	Helpful empty-state message appears	[Run test; record result]

5.1 Screenshot checklist

SMART PANTRY MANAGER

My Pantry

+ ADD INGREDIENT

FIND MATCHING RECIPES

egg — 2.0 each DELETE

oil — 5.0 ml DELETE

tomato — 1.0 each DELETE

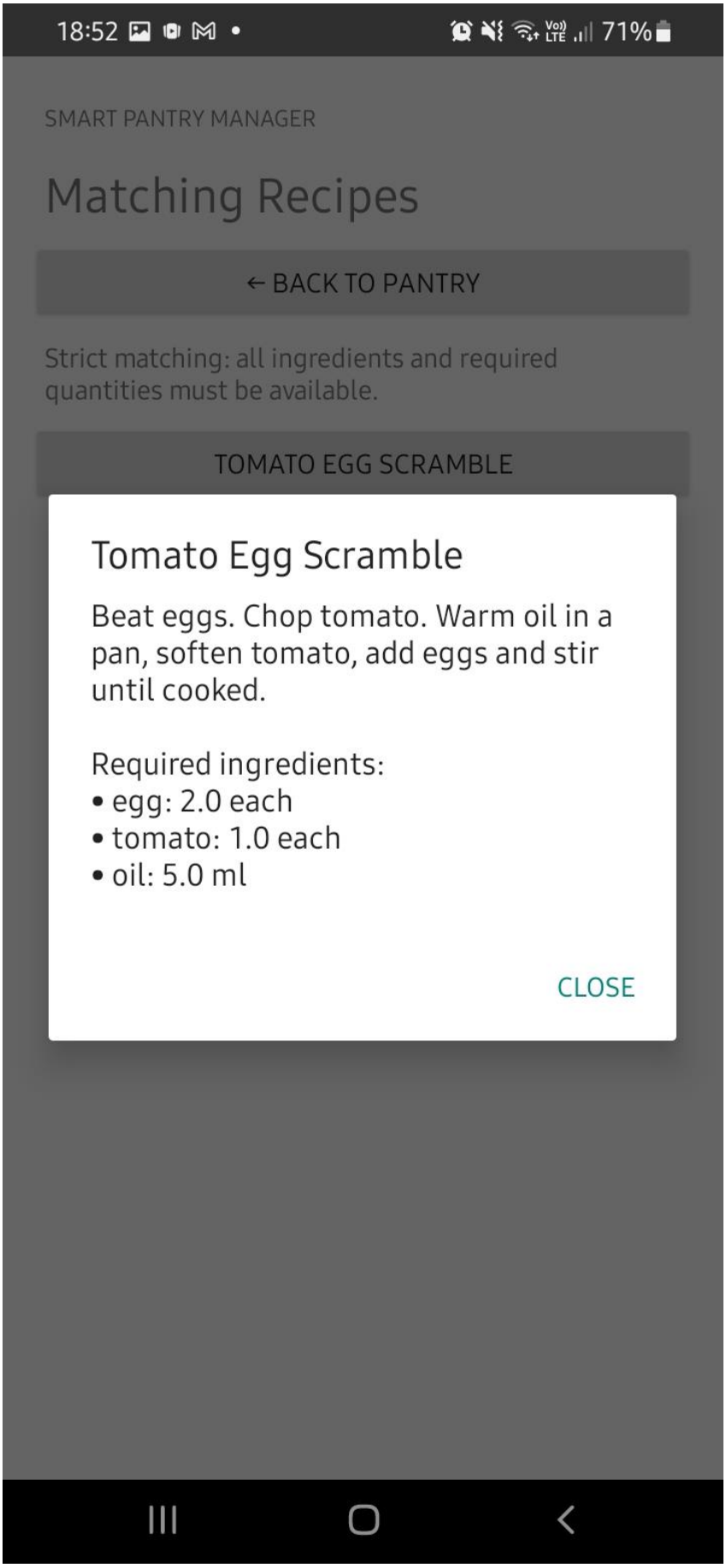
SMART PANTRY MANAGER

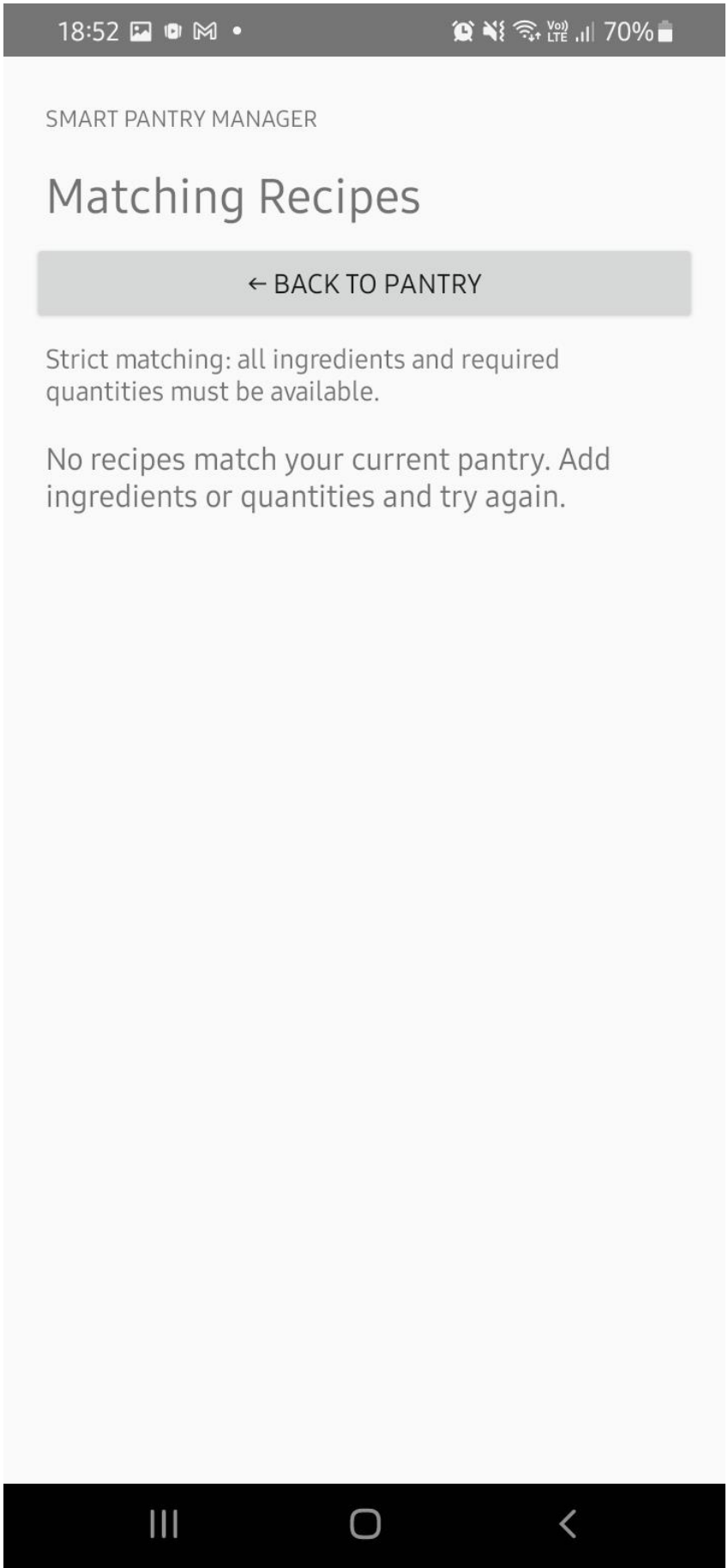
Matching Recipes

← BACK TO PANTRY

Strict matching: all ingredients and required quantities must be available.

TOMATO EGG SCRAMBLE


18:52
71%
SMART PANTRY MANAGER
Matching Recipes
← BACK TO PANTRY
Strict matching: all ingredients and required quantities must be available.
TOMATO EGG SCRAMBLE
Tomato Egg Scramble
Beat eggs. Chop tomato. Warm oil in a pan, soften tomato, add eggs and stir until cooked.
Required ingredients:
• egg: 2.0 each
• tomato: 1.0 each
• oil: 5.0 ml
CLOSE


18:52
70%
SMART PANTRY MANAGER
Matching Recipes
← BACK TO PANTRY
Strict matching: all ingredients and required quantities must be available.
No recipes match your current pantry. Add ingredients or quantities and try again.

6. Challenges and Solutions

Replace the examples below with the real challenges you encountered and the solutions you actually applied. If a challenge was not encountered, remove it rather than claiming it happened.

6.1 Strict matching and ingredient names

Potential challenge: ingredient names may differ in capitalisation or whitespace, such as “Tomato” and “tomato”. A practical solution is to normalise names before comparison and document any limits of singular/plural matching.

6.2 Persistent storage

Potential challenge: data can disappear if records are stored only in temporary in-memory collections. A persistent database resolves this by saving records to local or cloud storage and loading them when the application starts.

6.3 User input validation

Potential challenge: blank names or invalid quantities can cause unreliable data. Validate the form before writing to the database and provide a clear message explaining what the user needs to correct.

7. Conclusion and Reflection

Smart Pantry Manager addresses household food waste by connecting pantry tracking with recipe discovery. Its most important requirement is that a suggested recipe must be achievable using the ingredients and quantities already recorded by the user. CRUD operations, persistent storage, clear navigation and helpful validation support the core experience.

After completing and testing the implementation, this section should be updated to reflect what was learned about Java, Android Activities, Intents, adapters, database persistence and business logic. Future improvements could include more robust ingredient synonym handling, expiry reminders and accessibility refinements, provided they do not weaken the strict-matching rule.

8. References

Android Developers (n.d.) *Activities*. Available at: https://developer.android.com/guide/components/activities (Accessed: 23 October 2026).

Android Developers (n.d.) *Save data using SQLite*. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 23 October 2026).

Android Developers (n.d.) *Create dynamic lists with RecyclerView*. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 23 October 2026).

Oracle (n.d.) *Java Documentation*. Available at: https://docs.oracle.com/en/java/ (Accessed: 23 October 2026).